

Assignment Decision Trees and Ensemble Methods

Assigned: October 8, 2026
Due: on Canvas, October 22, 2026, 11:59PM
Points: 100 (+10 bonus, see below)

Overview

Build single CART trees and six families of tree ensembles, then compare them on accuracy, variance, training cost and interpretability using one classification and one regression dataset.

Learning objectives. By the end, students should be able to:

- 1. Explain how CART chooses splits, and how Gini impurity, entropy (information gain) and squared error differ.
- 2. Diagnose over- and under-fitting in a tree via depth, leaf size and cost-complexity pruning.
- 3. Explain why averaging reduces variance (bagging, pasting, random subspaces, random patches, random forests, extra trees).
- 4. Explain why sequential re-weighting reduces bias (AdaBoost, gradient boosting).
- 5. Design a fair experiment: fixed splits, seeds, cross-validation, and repeated runs with error bars.

Tools. Python with scikit-learn, or MATLAB with the Statistics and Machine Learning Toolbox. Students use library implementations for fitting, but must write their own experiment harness, plots and analysis. A bonus task asks them to implement one piece from scratch.

Format Teams of 1–3, time 2 weeks, submitted as a notebook or live script plus a report of at most 10 pages.

Tool mapping.

Method	scikit-learn	MATLAB
Classification tree	DecisionTreeClassifier(criterion="gini" / "entropy")	fitctree(..., 'deviance' / 'entropy')
Regression tree	DecisionTreeRegressor(criterion="squared_error" / "absolute_error" / "friedman_mse")	fitrtree
Pruning	ccp_alpha , cost_complexity_pruning_path	prune , cvloss
Bagging / pasting	BaggingClassifier(bootstrap=True / False)	fitcensemble(. TreeBagger

Method	scikit-learn	MATLAB
Random subspaces	<code>BaggingClassifier(max_features<1, bootstrap_features=False, max_samples=1.0)</code>	<code>fitcensemble(. (kNN/discriminant with NumPredictors</code>
Random patches	<code>BaggingClassifier(max_samples<1, max_features<1)</code>	<code>TreeBagger with NumPredictorsT</code>
Random forest	<code>RandomForestClassifier / Regressor</code>	<code>TreeBagger , or templateTree('I</code>
Extra trees	<code>ExtraTreesClassifier</code>	not built in (option
AdaBoost	<code>AdaBoostClassifier (SAMME)</code>	<code>fitcensemble(. (multiclass)</code>
Gradient boosting	<code>HistGradientBoostingClassifier , GradientBoostingRegressor</code>	<code>fitcensemble(. fitrensemble(. </code>
Voting / stacking	<code>VotingClassifier , StackingClassifier</code>	combine prediction

Datasets

Use **Fashion-MNIST** for classification and **California Housing** for regression. MNIST works, but it is close to saturated: a random forest reaches roughly 97% with little tuning, so differences between ensembles shrink to a fraction of a percent and are hard to interpret. Fashion-MNIST has the same format (70,000 28×28 grayscale images, 10 classes) but is harder, so single trees land near 80% and ensembles spread out across a visibly wider range. Its confusable classes (shirt vs. T-shirt vs. pullover) also make confusion matrices worth discussing.

Pixel data also makes feature importance visual: a random forest's importances reshaped to 28×28 form a heat map of where the model looks. That is a memorable result for students.

Dataset	Task	Size	Why use it	Load
Fashion-MNIST (primary)	10-class classification	70,000 × 784	Harder than MNIST; importance heat maps; class confusions	<code>fetch_openml('Fashion-MNIST')</code>
MNIST (alternative)	10-class classification	70,000 × 784	Familiar; good as a warm-up or comparison	<code>fetch_openml('mnist_784')</code>

Dataset	Task	Size	Why use it	Load
California Housing	Regression	20,640 × 8	Small, tabular, interpretable splits (income, location)	<code>fetch_california_housing()</code>
Covertypes (optional)	7-class, imbalanced	581,012 × 54	Tabular data where trees shine; class imbalance	<code>fetch_covtype()</code>

MATLAB users can download the same files from OpenML or the Fashion-MNIST GitHub repository and read them with `readmatrix`; California Housing is also widely available as a CSV file.

Compute budget. AdaBoost and repeated sweeps on 60,000 × 784 images are slow on laptops. Require a fixed stratified subsample: 10,000 training images, 5,000 validation, and the official 10,000 test set touched only once at the end. Optional: compare against PCA to 50 components to show how ensembles behave on dense, rotated features.

Part 1 — Single CART trees (20 pts)

For a node with class proportions p_k , students compare these impurity measures; the split chosen maximizes the weighted decrease in impurity.

$$G = 1 - \sum_k p_k^2 \quad H = -\sum_k p_k \log_2 p_k \quad \Delta = I(\text{parent}) - \sum_c \frac{n_c}{n} I(c)$$

1a. Warm-up by hand (3 pts). For a node with 40 samples (25 class A, 15 class B) and a candidate split into (20 A, 2 B) and (5 A, 13 B), compute the Gini decrease and the information gain. Then plot G , $H/2$ and misclassification error as functions of p for two classes, and explain why CART prefers G or H over misclassification error.

1b. Criterion comparison (7 pts). Train classification trees on Fashion-MNIST with Gini and with entropy (MATLAB: also `twining`). For each criterion, sweep `max_depth` $\in \{2, 4, 6, 8, 10, 12, 15, 20, \text{None}\}$ and report:

- train and validation accuracy vs. depth (one plot, both criteria);
- training time, number of leaves and actual depth;
- how often the two criteria choose the same feature at the root and at depth 1–2;
- mean $\pm$ std over 5 random seeds or subsamples.

State whether the criterion matters as much as depth does. (Expected finding: usually it does not; the difference is small next to the effect of depth.)

1c. Pruning (5 pts). Compute the cost-complexity pruning path (`ccp_alpha` ; MATLAB `prune` with `cvloss`). Plot validation accuracy and leaf count vs. α , choose α by 5-fold CV, and compare against the best depth-limited tree. Also try `min_samples_leaf` $\in \{1, 5, 20, 50\}$.

1d. Regression trees (5 pts). On California Housing, compare `squared_error`, `absolute_error` and `friedman_mse`. Report RMSE and MAE vs. depth. Visualize the top three levels of the best tree and interpret the first splits in plain language. Show the step-function nature of the prediction by plotting predicted vs. actual price for one feature (e.g., median income).

Part 2 — Sampling-based ensembles (25 pts)

All four methods below train trees on random subsets of rows, columns, or both. They differ only in what is sampled, so students should implement all four with one `BaggingClassifier` (or `TreeBagger`) setup and change only the sampling arguments.

Method	Rows (samples)	Columns (features)	Reference
Pasting	subset, without replacement	all	Breiman, 1999
Bagging	bootstrap, with replacement	all	Breiman, 1996
Random subspaces	all	subset, fixed per tree	Ho, 1998
Random patches	subset	subset, fixed per tree	Louppe & Geurts, 2012

2a (8 pts). For each method, use 100 fully grown trees. Sweep the sampled fraction of rows and/or columns over $\{0.1, 0.3, 0.5, 0.7, 1.0\}$. For random patches, show a heat map of validation accuracy over the (row fraction $\times$ column fraction) grid.

2b (6 pts). Plot validation accuracy vs. number of trees (1 to 300) for bagging. On the same plot, show the out-of-bag (OOB) estimate (`oob_score=True` ; MATLAB `oobError`). How close is OOB to validation accuracy, and why does pasting not have one?

2c (6 pts). Bias–variance. Train each method on 10 different bootstrap resamples of the training set. For each test point, measure how often the 10 models disagree with each other and how often their majority vote is wrong. Show that averaging reduces disagreement

(variance) far more than it reduces error from systematic bias. Compare against a single unpruned tree and a depth-4 tree.

2d (5 pts). Repeat 2a with depth-limited base trees (depth 4). Explain why bagging helps deep trees much more than shallow ones.

Part 3 — Random forests and extra trees (15 pts)

A random forest is bagging plus a fresh random feature subset at every split, rather than one per tree. Extra trees go further and also randomize the split thresholds.

3a (6 pts). Sweep `max_features` $\in \{1, \sqrt{p}, \log_2 p, 0.1p, 0.3p, p\}$ with 300 trees. Note that `max_features=p` is plain bagging. Plot validation accuracy and training time. Compare Gini vs. entropy once more inside the forest.

3b (4 pts). Compare random forest vs. extra trees on accuracy, training time, and the variance across 5 seeds.

3c (5 pts). Feature importance. Reshape impurity-based importances to 28×28 and display them as a heat map. Do the same with permutation importance on the validation set. Explain the difference, and why impurity importance can be biased toward high-cardinality features. For California Housing, show a bar chart of both importances.

Part 4 — Boosting (25 pts)

Boosting fits trees sequentially, each focused on the previous ensemble's errors. Students should contrast it with Parts 2–3: boosting attacks bias, averaging attacks variance.

4a. AdaBoost (10 pts). Use SAMME (`AdaBoostClassifier` ; MATLAB `AdaBoostM2`) with base trees of depth 1 (stumps), 2, 4 and 8. Use up to 500 rounds and learning rates $\{0.1, 0.5, 1.0\}$. Plot train and validation error vs. round (`staged_predict` ; MATLAB `loss` with `'Mode', 'cumulative'`). Answer: do stumps work on 10-class image data? Does validation error keep falling after training error reaches zero?

4b. Sample weights (5 pts). After 50 rounds, show the 20 training images with the highest AdaBoost weights. Are they mislabeled, ambiguous, or just hard? Briefly connect this to AdaBoost's sensitivity to label noise. Then flip 10% of training labels at random and rerun AdaBoost and a random forest; compare the drop in accuracy.

4c. Gradient boosting (10 pts). Train `HistGradientBoostingClassifier` (MATLAB: `LogitBoost` or `GentleBoost`) and `GradientBoostingRegressor` on California Housing (MATLAB: `LSBoost`). Tune learning rate, depth and number of rounds with early stopping. Show the learning-rate vs. rounds trade-off on one plot. For regression, also run `AdaBoost.R2` (`AdaBoostRegressor`) and compare.

Part 5 — Combining classifiers (5 pts)

Build a hard-voting and a soft-voting ensemble from the best single tree, best random forest, best AdaBoost and best gradient boosting model. Then build a stacking ensemble with logistic regression as the meta-learner. Does combining beat the best single member? Measure how correlated the members' errors are, and use that to explain the result.

Part 6 — Final comparison and report (10 pts)

Evaluate their best configuration of every method once on the untouched test set and fill in this table (mean $\pm$ std over 5 seeds):

Method	Best hyperparameters	Test accuracy (%)	Macro-F1	Train time (s)	Predict time (ms / 1k images)	Model size (nodes)
Tree (Gini)						
Tree (entropy)						
Pasting						
Bagging						
Random subspaces						
Random patches						
Random forest						
Extra trees						
AdaBoost						
Gradient boosting						
Voting / stacking						

Plus the same table with RMSE and MAE for California Housing. The report must also include a confusion matrix for the best model, a short paragraph per part, and a one-page conclusion recommending a method for (a) best accuracy, (b) a tight compute budget, and (c) when the model must be explained to a non-expert.

Deliverables

- ☐ Notebook (`.ipynb`) or MATLAB live script (`.mlx`) that runs end to end with a fixed random seed
- ☐ PDF report, at most 10 pages, every figure with labeled axes and a caption
- ☐ Results tables above, as CSV
- ☐ A one-paragraph statement of who did what (teams)

Grading rubric

Part	Points	Full credit requires
1 — Single trees	20	Correct hand computation; criterion and depth sweeps with error bars; pruning chosen by CV
2 — Sampling ensembles	25	All four methods from one shared setup; OOB vs. validation; a bias–variance argument backed by measurements
3 — Forests and extra trees	15	<code>max_features</code> sweep; importance heat maps; impurity vs. permutation discussed
4 — Boosting	25	Staged error curves; label-noise experiment; early stopping used correctly
5 — Combining	5	Voting and stacking compared; error correlation measured
6 — Comparison and report	10	Test set used once; clear, justified recommendations
Bonus	+10	See below

Across all parts, deduct up to 10 points for tuning on the test set, single-run results with no variance estimate, or unlabeled plots.

Bonus (+10). Implement one of the following from scratch in NumPy or MATLAB (no library tree code), and check it against the library on a 2-feature toy dataset: (a) a CART tree with Gini and entropy, depth and leaf-size limits; or (b) multiclass SAMME AdaBoost on top of the library's stumps, logging the sample weights at every round.